

ACADEMIC REPORT: SMART ROBOTIC CAR CONTROLLER

Embedded Systems Control and Telemetry Data Pipeline

Course: Advanced Programming & Real-Time Control Systems

Prepared By: Mostafa Abdelzaher

Department: Electrical Engineering (Computers & Control Division)

Institution: Higher Institute of Engineering & Technology, Kafr El-Sheikh, Egypt

1. Executive Summary

This academic report presents the detailed architecture, low-level register configuration, and firmware design of an advanced robotic car controller built on the 8-bit AVR RISC architecture utilizing the ATmega32 microcontroller. Operating at an external crystal frequency of 16 MHz, the system integrates a safety-critical real-time control system with a live telemetry pipeline. By avoiding blocking designs and using optimized bitwise masking, the controller manages DC motor motion through dual-channel Fast PWM, reads ambient temperature via an integrated ADC, and monitors obstacles using an ultrasonic safety envelope. Telemetry packets are structured dynamically and streamed via UART to a mobile device, bridging the gap between hardware control and streaming data engineering.

2. Low-Level Register Configurations

Direct hardware manipulation requires configuring the ATmega32 physical registers. Below are the bitwise layouts and configurations for the active peripherals in this project.

2.1. UART Configuration Registers

The Universal Asynchronous Receiver and Transmitter (UART) peripheral uses multiple control, status, and baud rate registers.

UCSRA – USART Control and Status Register A

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
RXC	TXC	UDRE	FE	DOR	PE	U2X	MPCM

- **RXC (USART Receive Complete - Bit 7):** Hardware sets this bit to 1 when there are unread data in the receive buffer. It is used in UART_Receive() to detect incoming commands.
- **UDRE (USART Data Register Empty - Bit 5):** Indicates if the transmit buffer is ready to receive new data. Hardware sets this to 1 when the transmit buffer is empty. Used in UART_SendChar().

UCSRB – USART Control and Status Register B

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
RXCIE	TXCIE	UDRIE	RXEN	TXEN	UCSZ2	RXB8	TXB8

- **RXEN (Receiver Enable - Bit 4):** Writing 1 to this bit enables the UART Receiver, redirecting Pin 14 (PD0/RXD) to the UART block.
- **TXEN (Transmitter Enable - Bit 3):** Writing 1 to this bit enables the UART Transmitter, redirecting Pin 15 (PD1/TXD) to the UART block.

UCSRC – USART Control and Status Register C

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
URSEL	UMSEL	UPM1	UPM0	USBS	UCSZ1	UCSZ0	UCPOL

- **URSEL (Register Select - Bit 7):** Must be written to 1 to select writing to the UCSRC register instead of the UBRRH register, which share the same I/O memory address.
- **UCSZ1:0 (Character Size - Bits 2 & 1):** Configured to '11' to select an 8-bit data frame length, standard for most Bluetooth and serial terminals.

2.2. Analog-to-Digital Converter (ADC) Registers

The ADC converts analog voltage levels from the LM35 Temperature Sensor to digital representations.

ADMUX – ADC Multiplexer Selection Register

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
REFS1	REFS0	ADLAR	MUX4	MUX3	MUX2	MUX1	MUX0

- **REFS0 (Reference Selection - Bit 6):** Setting this bit to 1 selects AVCC with an external capacitor at the AREF pin as the voltage reference ($V_{REF} = 5.0V$).
- **MUX2:0 (Analog Channel Selection - Bits 2:0):** Modified dynamically during runtime to select the analog input channel (specifically set to 110 to read from ADC Channel 6 on PA6).

ADCSRA – ADC Control and Status Register A

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
ADEN	ADSC	ADATE	ADIF	ADIE	ADPS2	ADPS1	ADPS0

- **ADEN (ADC Enable - Bit 7):** Enables the ADC hardware block. Must be written to 1 before any conversion can start.
- **ADSC (ADC Start Conversion - Bit 6):** Write 1 to start a conversion. It remains 1 during conversion and is cleared by hardware when the process completes.
- **ADPS2:0 (ADC Prescaler Select - Bits 2:0):** Configured to '111' to establish a division factor of 64 between the CPU clock (16 MHz) and the ADC clock ($16\text{ MHz} / 64 = 250\text{ kHz}$).

2.3. Timer1 (PWM Generation) Registers

Timer1 is configured as an 8-bit Fast PWM generator to output analog-equivalent voltage levels on Pins PD4 and PD5, driving the L298N speed inputs.

TCCR1A – Timer/Counter 1 Control Register A

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
COM1A1	COM1A0	COM1B1	COM1B0	FOC1A	FOC1B	WGM11	WGM10

- **COM1A1 & COM1B1 (Compare Output Mode - Bits 7 & 5):** Set to 1 to configure both channel A and B in Non-Inverting Fast PWM. The outputs (PD5/PD4) clear on compare match and set at BOTTOM.
- **WGM10 (Waveform Generation Mode - Bit 0):** Combined with WGM12 in TCCR1B, it configures Mode 5 (Fast PWM, 8-bit resolution, TOP = 0x00FF).

TCCR1B – Timer/Counter 1 Control Register B

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
ICNC1	ICES1	-	WGM13	WGM12	CS12	CS11	CS10

- **WGM12 (Waveform Generation Mode - Bit 3):** Written to 1 to complete Fast PWM 8-bit mode configuration.
- **CS11 & CS10 (Clock Select - Bits 1 & 0):** Written to '011' to select a prescaler of 64 from the system clock ($16\text{ MHz} / 64 = 250\text{ kHz}$ clock rate).

3. Firmware Implementation & Code Explanation

The entire controller code is structured into highly cohesive modules representing different hardware functionalities. Below is the block-by-block logical analysis.

3.1. Main Declarations & Header Imports

```
#ifndef F_CPU
#define F_CPU 16000000UL
#endif

#include <avr/io.h>
#include <util/delay.h>
#include <stdio.h>
```

The system clock rate is defined as 16 MHz with UL (Unsigned Long) to prevent integer overflow during math evaluations. <avr/io.h> maps memory addresses to human-readable register definitions, <util/delay.h> provides precise hardware-locked software loops for delays, and <stdio.h> enables formatting telemetry strings via `sprintf()`.

3.2. Serial Communication Interface (UART)

```
void UART_Init(void) {
    UBRRL = 103; // 9600 Baud Rate @ 16MHz
    UBRRH = 0;
    UCSRB = (1 << RXEN) | (1 << TXEN);
    UCSRC = (1 << URSEL) | (1 << UCSZ1) | (1 << UCSZ0);
}

void UART_SendChar(char data) {
    while (!(UCSRA & (1 << UDRE)));
    UDR = data;
}

void UART_SendString(char* str) {
    while (*str) UART_SendChar(*str++);
}

char UART_Receive(void) {
    if (UCSRA & (1 << RXC)) return UDR;
    return 0;
}
```

UART_Init() maps the Baud Rate register to 103 (achieved by dividing the clock to 9600 bps with an error rate of 0.16%). It enables receiver and transmitter circuits and structures the frame format to an asynchronous 8-bit frame.

UART_SendChar() uses a Polling architecture to block further processing until the transmit register buffer becomes empty, then loads the target byte onto UDR.

UART_Receive() is built using a non-blocking if check instead of a blocking while loop. This allows the system to continue calculating obstacle proximity without hanging the main execution

flow if there are no new commands.

3.3. Analog Signal Extraction (ADC)

```
void ADC_Init(void) {
    ADMUX = (1 << REFS0);
    ADCSRA = (1 << ADEN) | (1 << ADPS2) | (1 << ADPS1) | (1 << ADPS0);
}

uint16_t ADC_Read(uint8_t channel) {
    ADMUX = (ADMUX & 0xF8) | (channel & 0x07);
    ADCSRA |= (1 << ADSC);
    while (ADCSRA & (1 << ADSC));
    return ADC;
}
```

ADC_Init() routes \$V_{REF}\$ to AVCC and locks the conversion frequency to 250 kHz for speed and responsiveness.

ADC_Read() masks the selection channel via (channel & 0x07) to protect critical configuration registers against invalid channel values. It fires the conversion using ADSC and monitors the status bit until hardware finishes the digitization process and clears it, returning a value between 0 and 1023.

3.4. Ultrasonic Range Finding & Collision Prevention

```
void Ultrasonic_Init(void) {
    DDRA |= (1 << PA4); // Trig Output
    DDRA &= ~(1 << PA5); // Echo Input
}

uint16_t Get_Distance(void) {
    uint16_t count = 0;
    uint16_t timeout = 0;

    PORTA &= ~(1 << PA4);
    _delay_us(2);
    PORTA |= (1 << PA4);
    _delay_us(10);
    PORTA &= ~(1 << PA4);

    while (!(PINA & (1 << PA5))) {
        timeout++;
        _delay_us(1);
        if (timeout > 30000) return 999;
    }

    while (PINA & (1 << PA5)) {
```

```

        count++;
        _delay_us(1);
        if (count > 30000) break;
    }
    return (uint16_t)(count * 0.034 / 2);
}

```

Calculates distance by triggering a high logic level pulse for exactly 10 microseconds on the trigger pin. To prevent the system from getting stuck due to sensor hardware failures or disconnected wires, a software-based timeout condition is established. When the echo pin transitions to high, the controller tracks the round-trip signal travel time, converting it to distance in centimeters based on the speed of sound ($0.034 \text{ cm}/\mu\text{s}$).

3.5. Dual-Channel Speed Controls & Dynamic Driving Inputs

```

void PWM_Init(void) {
    DDRD |= (1 << PD4) | (1 << PD5);
    TCCR1A = (1 << COM1A1) | (1 << COM1B1) | (1 << WGM10);
    TCCR1B = (1 << WGM12) | (1 << CS11) | (1 << CS10); // Prescaler 64
}

void Motor_Init(void) {
    DDRA |= 0x0F; // PA0 to PA3 as Output
    PORTA &= 0xF0; // Reset motors
}

```

PWM_Init() maps pins PD4 and PD5 as Fast PWM outputs under Timer1 at a carrier frequency of approximately 976 Hz. This frequency is high enough to eliminate mechanical motor humming while minimizing heat dissipation in the L298N driver transistors.

Motor_Init() configures pins PA0 to PA3 as digital outputs to drive the L298N direction inputs.

3.6. Control Integration Loop & Real-Time Telemetry

```

int main(void) {
    UART_Init();
    PWM_Init();
    Motor_Init();
    Ultrasonic_Init();
    ADC_Init();

    OCR1A = 255;
    OCR1B = 255;

    char command;
    char buffer[32];
    uint16_t distance;
}

```

```

uint16_t temp_c;

while (1) {
    distance = Get_Distance();
    temp_c = (uint32_t)ADC_Read(6) * 500 / 1024;

    sprintf(buffer, "D:%d,T:%d#", distance, temp_c);
    UART_SendString(buffer);

    if (distance > 0 && distance < 15) {
        PORTA &= 0xF0;
    }

    command = UART_Receive();
    if (command != 0) {
        switch (command) {
            case 'F': PORTA = (PORTA & 0xF0) | 0b00000101; break; // Forward
            case 'B': PORTA = (PORTA & 0xF0) | 0b00001010; break; // Backward
            case 'R': PORTA = (PORTA & 0xF0) | 0b00000110; break; // Right
            case 'L': PORTA = (PORTA & 0xF0) | 0b00001001; break; // Left
            case 'S': PORTA = (PORTA & 0xF0) | 0x00; break; // Stop
        }
    }
    _delay_ms(50);
}
return 0;
}

```

The main super-loop unifies all hardware modules. First, sensor readings are taken and packaged into a standardized telemetry string (e.g., "D:24,T:28#") and sent over UART. Next, the safety check validates the proximity threshold. If the distance falls below 15 cm, the controller overrides user inputs and stops the motors. User commands are parsed using a protective bitwise mask ($\text{PORTA} \& 0xF0$) to alter only the driving pins (PA0-PA3) while keeping the ultrasonic pins (PA4-PA5) intact.

4. Mathematical Proofs and Derivations

To validate the reliability of this real-time system, we derive the exact relationships for serial baud rate timing, ADC voltage extraction, and PWM carrier frequencies.

4.1. Baud Rate Calculation

For standard asynchronous serial communication, the register value \$UBRR\$ is calculated using the following formula:

$$UBRR = (F_{CPU} / (16 * Baud)) - 1$$

Given $F_{CPU} = 16,000,000 \text{ Hz}$ and $Baud = 9600 \text{ bps}$:

$$UBRR = (16,000,000 / (16 * 9600)) - 1 = 104.16 - 1 = 103.16 \approx 103$$

Evaluating the actual baud rate with $UBRR = 103$:

$$Actual\ Baud = 16,000,000 / (16 * 104) = 9615.38 \text{ bps}$$

The resulting error is:

$$Error\ \% = | (9615.38 - 9600) / 9600 | * 100\% = 0.16\%$$

This minimal error rate (well below the maximum allowable threshold of 2%) guarantees reliable communication with the HC-05 Bluetooth transceiver without framing issues.

4.2. LM35 Voltage to Temperature Conversion

The ATmega32 ADC uses a 10-bit successive approximation register, mapping input voltages linearly to digital values from 0 to 1023. The voltage step size is:

$$Resolution\ Step = V_{REF} / 1024 = 5.0V / 1024 = 4.88 \text{ mV}$$

The LM35 sensor provides an output of $10 \text{ mV}/^{\circ}\text{C}$. The relation between Celsius temperature (T) and sensor output voltage (V_{out}) is:

$$V_{out} = T * 0.01V$$

Equating this to the digital ADC reading (ADC_Val):

$$ADC_Val = (V_{out} / V_{REF}) * 1024 = (T * 0.01 / 5.0) * 1024$$

Solving for T yields:

$$T = ADC_Val * (5.0 / (1024 * 0.01)) = ADC_Val * (500 / 1024)$$

This formula is implemented in firmware to calculate the temperature without relying on slow float-point arithmetic.

4.3. PWM Frequency Generation

For 8-bit Fast PWM, the counter counts from 0 to 255 before resetting. The output frequency is:

$$f_{PWM} = F_{CPU} / (N * 256)$$

With prescaler \$N = 64\$:

$$f_{PWM} = 16,000,000 / (64 * 256) = 16,000,000 / 16384 = 976.56 \text{ Hz}$$

This frequency is optimal for standard DC motor control, balancing responsiveness against switching efficiency.

5. Recommendations for Industrial Deployment

To upgrade this system for industrial deployment, we suggest implementing the following hardware and software improvements:

1. **Input Capture Unit (ICU) Integration:** Instead of calculating ultrasonic travel times using blocking software polling loops, route the Echo pin to PD6 (ICP1) to utilize the hardware input capture unit of Timer1. This allows the system to measure distances in the background, freeing up CPU overhead.
2. **Hardware Noise Decoupling:** Inductive motor spikes can cause voltage drops and EMI. Place 0.1 μF decoupling ceramic capacitors across the motor terminals and close to the ATmega32 power supply pins to suppress high-frequency noise and prevent unexpected controller resets.